Politechnika Wrocławska
Wydział Elektroniki, Fotoniki i Mikrosystemów

KIERUNEK: Inteligentna Elektronika

PRACA DYPLOMOWA INŻYNIERSKA

Tytuł pracy:
Opracowanie aplikacji do analizy statystycznej obrazów SEM samoorganizujących się nanostruktur metalicznych na podłożach SiO_2/GaN

AUTOR:
Bartosz Makowski

OPIEKUN:
dr inż. Andrzej Stafiniak

WROCŁAW 2025

Streszczenie

Niniejsza praca przedstawia aplikację do analizy statystycznej obrazów SEM samoorganizujących się nanostruktur metalicznych, zbudowaną w języku *Python* z wykorzystaniem biblioteki *PyImageJ*. Opracowane narzędzie automatyzuje cały proces przetwarzania: wczytanie obrazu, wstępną obróbkę, segmentację, pomiary topologiczne oraz generowanie wykresów i plików *.csv* z wynikami i informacjami statystycznymi. Aplikacja wyznacza parametry topologiczne dla każdej wykrytej struktury, w tym pole powierzchni, promień, okrągłość, średnicę Fereta oraz współrzędne centroidu, a także oblicza statystyki globalne, takie jak średnie, mediany, odchylenia standardowe, pokrycie powierzchni czy gęstość powierzchniowa cząstek. Rozwiązanie zostało dostosowane do potrzeb *Katedry Mikroelektroniki i Nanotechnologii*, w celu analizy samoorganizujących się nanostruktur metalicznych na podłożach SiO_2/GaN, skracając czas analizy i zwiększając jej powtarzalność przy charakteryzacji.

This paper presents an application for statistical analysis of SEM images of self-organizing metallic nanostructures, built in *Python* using the *PyImageJ* library. The developed tool automates the entire processing workflow: image loading, preprocessing, segmentation, topological measurements, and generation of graphs and *.csv* files with results and statistical information. The application determines the topological parameters for each detected structure, including surface area, radius, roundness, Feret diameter, and centroid coordinates, and calculates global statistics such as mean, median, standard deviation, surface coverage and particle surface density. The solution has been tailored to the needs of the *Department of Microelectronics and Nanotechnology* in order to analyze self-organizing metallic nanostructures on SiO_2/GaN substrates, reducing analysis time and increasing repeatability in the characterization.

Badania współfinansowane przez:
Narodowe Centrum Nauki - projekt OPUS, 2022/45/B/ST5/04292,
Narodowe Centrum Badań i Rozwoju – projekt 36/564935/SPUB/SP/2023,
Narodową Agencję Wymiany Akademickiej - umowa BPN/BSK/2023/1/00040/U/00001,
oraz w ramach subwencji Politechniki Wrocławskiej.

Badania zostały zrealizowane dzięki wskaźnikom produktu i wskaźnikom rezultatu osiągniętym w ramach projektu *Narodowego Centrum Badań i Rozwoju TECHMATSTRATEG 1/346922/4/NCBR/2017.*

Dziękuje bliskim za wsparcie i wiarę we mnie,
dr. inż. Andrzejowi Stafiniakowi za pomoc merytoryczną,
społeczności akademickiej Politechniki Wrocławskiej, prowadzącym,
koleżankom i kolegom, za możliwość rozwoju i niezapomniane siedem semestrów.

Spis treści

1. Wykaz skrótów i oznaczeń . . . 4
2. Wstęp . . . 5
3. Wprowadzenie teoretyczne . . . 7
3.1. SEM . . . 7
3.1.1. Zasada działania . . . 8
3.2. Nanocząstki . . . 9
3.3. ImageJ . . . 11
4. Prace własne . . . 12
4.1. Przygotowanie środowiska programistycznego . . . 12
4.1.1. Inicjalizacja . . . 12
4.2. Wczytanie obrazu . . . 13
4.3. Kadrowanie . . . 14
4.4. Wstępna obróbka obrazu . . . 15
4.4.1. Wzmocnienie kontrastu . . . 15
4.4.2. Progowanie oraz binaryzacja . . . 16
4.4.3. Wypełnienie dziur . . . 17
4.5. Konfiguracja pomiaru . . . 17
4.5.1. Set Measurments . . . 17
4.5.2. setScale . . . 18
4.5.3. AnalyzeParticles . . . 20
4.6. Wyniki analizy . . . 21
4.6.1. Graficzne przedstawienie wyników . . . 22
4.7. Interfejs i obsługa aplikacji . . . 25
4.7.1. Praktyczne wykorzystanie aplikacji . . . 28
5. Podsumowanie i wnioski . . . 30
5.1. Podsumowanie . . . 30
5.2. Możliwości rozwoju . . . 31
5.3. Znane błędy i problemy . . . 31
6. Dodatki . . . 32
Spis rysunków . . . 32
Spis tabel . . . 32
Pełny kod aplikacji . . . 33

1. Wykaz skrótów i oznaczeń

SEM skaningowy mikroskop elektronowy (*ang. Scanning Electron Microscope*)
JVM wirtualna maszyna środowiska Java (*ang. Java Virtual Machine*)
ROI obszar roboczy (*ang. Region Of Interest*)
SE elektrony wtórne (*ang. Secondary Electron*)
BSE elektrony wstecznie rozproszone (*ang. Back-Scattered Electrons*)
SSD odwilżanie w stanie stałym (*ang. Solid-State Dewetting*)
GUI graficzny interfejs użytkownika (*ang. Graphical User Interface*)

W poniższej pracy *kursywą* w tekście zaznaczone są nazwy własne, a `czcionkę maszynową` wykorzystano do przedstawienia fragmentów kodu.

2. Wstęp

Celem pracy jest opracowanie aplikacji umożliwiającej analizę statystyczną obrazów SEM. Aplikacja ma stanowić narzędzie pozwalające na szybkie i powtarzalne wyznaczenie parametrów opisujących geometrię oraz rozkłady statystyczne struktur, minimalizując przy tym czynnik błędu ludzkiego.

Głównym elementem pracy będzie stworzenie skryptu wykorzystującego środowisko *ImageJ*, przy użyciu biblioteki *PyImageJ*. Skrypt ten ma umożliwić obsługę plików generowanych przez mikroskop SEM *Hitachi SU6600*.

Kolejnym celem jest stworzenie modułu odpowiedzialnego za obróbkę wstępną obrazu, skrypt ma wykonywać operacje takie jak **kadrowanie**, **wzmocnienie kontrastu** oraz **przekształcenie obrazu na czarno-białą maskę**. Na tym etapie, posługując się możliwościami biblioteki *PyImageJ*, aplikacja będzie wykorzystywać gotowe rozwiązania środowiska *ImageJ*.

Analiza geometryczna oraz eksport danych zostaną zaimplementowane do aplikacji poprzez automatyczne wyznaczanie podstawowych miar opisujących kształt i rozmiar struktur, na przykład **pola powierzchni, promień, okrągłość,** a także informacje statystyczne: **liczbę obiektów, średnie, mediany czy odchylenie standardowe**. Informacje te zostaną wyeksportowane do formatu *.csv*, pozwalając na dalszą analizę lub wizualizację.

Automatyczna wizualizacja danych połączona z archiwizacją jest ostatnim celem pracy. Aplikacja ma generować histogramy oraz inne graficzne formy prezentacji danych. Wizualizacja oparta będzie na bibliotekach *matplotlib* oraz *seaborn*, zapewniając czytelność i umożliwiając użytkownikowi szybką interpretację wyników. Wyniki zapisać będzie można w postaci graficznej, na przykład *.png*, jak i w formatach arkuszy, np. *.csv*.

Finalnym rezultatem pracy będzie w pełni funkcjonalna aplikacja umożliwiająca automatyczną analizę obrazów SEM. Stworzone narzędzie usprawni proces charakteryzacji struktur powierzchniowych, w tym np. wysp metalicznych formujących się w wyniku obróbki termicznej cienkich warstw metali, cząstek naniesionych na powierzchnię z zawiesin koloidalnych oraz kropek kwantowych wzrastanych metodami epitaksjalnymi.

Integracja etapów pozyskiwania, obróbki oraz wizualizacji danych pozwoli na znaczące skrócenie czasu analizy przy jednoczesnym zwiększeniu jej precyzji i powtarzalności.

Tematyka pracy wpisuje się w zakres profilu kierunku Inteligentnej elektroniki z uwagi na jej stosunkowo interdyscyplinarny charakter inżynierski. Praca łączy zagadnienia przetwarzania i analizy danych, których składowymi są statystyka oraz programowanie, z metodami pomiarowymi wykorzystującymi skaningową mikroskopię elektronową (SEM, ang. *Scanning Electron Microscopy*).

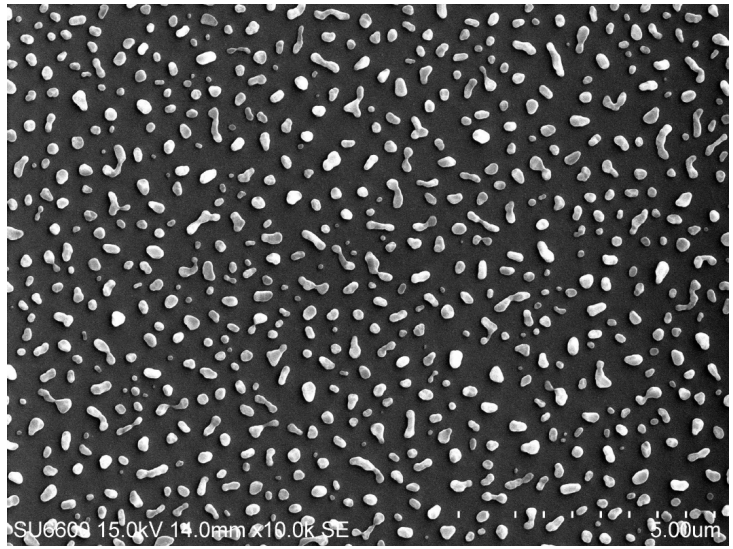
Opracowane narzędzie może znaleźć zastosowanie w pracach technologicznych prowadzonych na *Wydziale Elektroniki Mikrosystemów i Fotoniki PWr*, wspierając badania w zakresie nanotechnologii i charakteryzacji materiałów. Pierwszorzędnie funkcjonalność aplikacji dostosowano do potrzeb *Katedry Mikroelektroniki i Nanotechnologii*, jednak cała aplikacja zostanie napisana w sposób pozwalający na jej dalszą rozbudowę, a jej kod źródłowy udostępniony będzie na platformie *GitHub* pod linkiem: *https://github.com/BMakx/DyplomBartek*.

3. Wprowadzenie teoretyczne

3.1. SEM

Skaningowa mikroskopia elektronowa jest zaawansowaną techniką obrazowania wykorzystywaną do wizualizacji i analizy powierzchni materiałów z rozdzielczością dochodzącą do zakresu nanometrów.

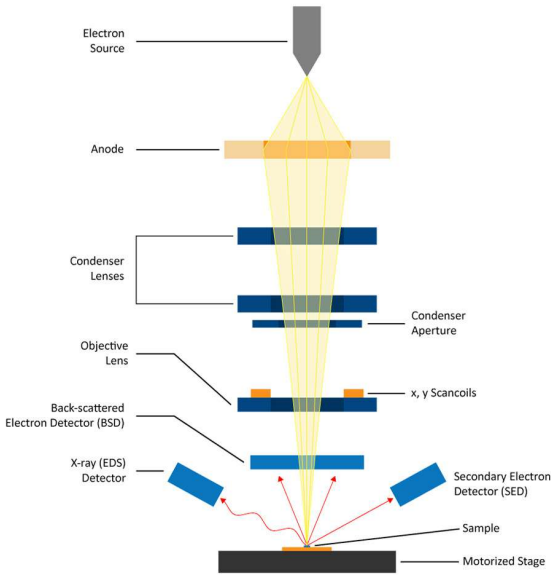
W porównaniu do mikroskopów optycznych, które wykorzystują światło widzialne, mikroskopy SEM używają wiązki elektronów o znacznie krótszej długości fali, co pozwala na uzyskanie obrazów o wielokrotnie wyższej rozdzielczości. Technika ta umożliwia szczegółową analizę topografii powierzchni oraz morfologii struktur. Mikroskopy SEM znajdują szerokie zastosowanie w naukach o materiałach, nanotechnologii, elektronice oraz badaniach biologicznych, pozwalając na charakteryzację struktur powierzchniowych w zakresie od mikrometrów do nanometrów [1]. Na rysunku 3.1 przedstawiono przykładowe zdjęcie z mikroskopu SEM.



Rys. 3.1: Przykładowy obraz z mikroskopu SEM *Hitachi SU6600* w trybie SE [2]

3.1.1. Zasada działania

Podstawą funkcjonowania mikroskopu SEM jest skanowanie powierzchni próbki skoncentrowaną wiązką elektronów generowaną przez źródło elektronowe. Elektrony są przyspieszane poprzez przyłożenie napięcia w zakresie od kilkuset woltów do kilkudziesięciu kilowoltów, a następnie ogniskowane i skanowane po powierzchni próbki [3]. Poniżej (Rys. 3.2) przedstawiono schemat budowy mikroskopu SEM.



Rys. 3.2: Budowa mikroskopu SEM [4]

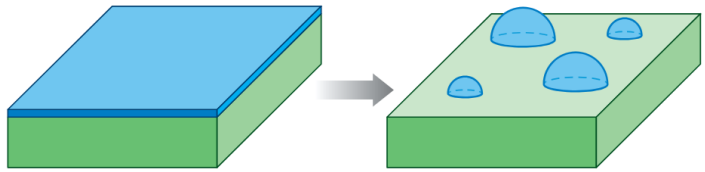
W wyniku oddziaływań elektronów z atomami analizowanej struktury generowane są różnorodne sygnały niosące informacje o badanym materiale. Kluczowe znaczenie mają dwa rodzaje elektronów: *elektrony wtórne* (SE, ang. *Secondary Electron*), czyli elektrony, które są wybite z badanej powierzchni, oraz *elektrony wstecznie rozproszone* (BSE, ang. *Back-Scattered Electrons*) będące elektronami odbitymi od badanej powierzchni.

Do badań w pracy wykorzystano obrazy z mikroskopu FE SEM (ang. *Field Effect Scanning Electron Microscope*) *Hitachi SU6600*. Mikroskop ten wyposażony jest w działo elektronowe w formie diody Schottki'ego na bazie materiałów W/ZrO_2. Taka konfiguracja umożliwia przeprowadzenie pomiarów przy zastosowaniu napięcia przyspieszającego do 30 kV, o maksymalnej rozdzielczości 1,2 nm.

3.2. Nanocząstki

Projektowana aplikacja może znaleźć zastosowanie w analizie obrazów SEM szerokiej gamy nanostruktur. Główną motywacją pracy jest wsparcie prac badawczych nad zjawiskiem samoorganizacji nanostruktur metalicznych.

Proces samoorganizacji cienkich warstw metali w odseparowane wyspy na powierzchni podłoża nazywany jest odwilżaniem w stanie stałym (SSD, ang. *solid-state dewetting*). Zjawisko to wynika z dążenia układu do minimalizacji całkowitej energii międzyfazowej na granicy metalu i podłoża (np. GaN, SiO_2). Prowadzi to do przekształcenia cienkiej warstwy w zbiory odizolowanych struktur, których kształt dąży do uzyskania korzystnego pod względem termodynamicznym profilu sferycznego. [5]. Proces ten poglądowo przedstawia rysunek 3.3.

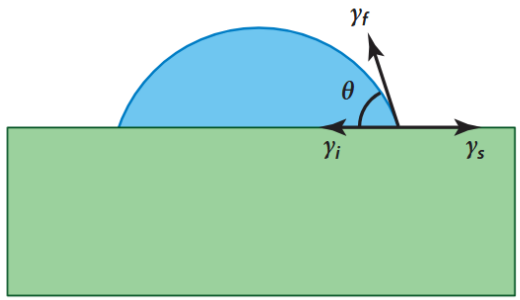


Rys. 3.3: Formowanie się wysp metalicznych [5]

Powstawanie wysp metalicznych zachodzi głównie poprzez dyfuzję powierzchniową, której celem jest obniżenie energii na granicach faz i osiągnięcie optymalnego kształtu. Dla wysp, których energia powierzchniowa jest równa w każdym kierunku, można przyjąć, że gęstość energii jest stała w całej ich objętości. Warunki równowagi na styku metal–podłoże–otoczenie można opisać *równaniem Younga–Dupré* (Rys. 3.4) [5]:

$$\gamma_s = \gamma_{is} + \gamma_i \cos\theta,$$

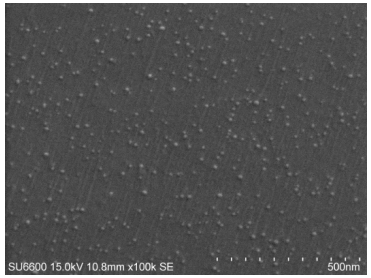
gdzie:
γ_{is} – energia powierzchniowa na granicy wyspa–podłoże,
γ_s – energia powierzchniowa podłoża,
γ_i – energia powierzchniowa wyspy,
θ – kąt równowagi między trzema fazami.



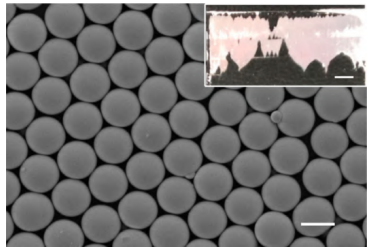
Rys. 3.4: W stanie równowagi wyspa przyjmuje kształt sferycznej czaszy o kącie równowagi θ. [5]

Prócz wysp metalicznych aplikacja umożliwia również analizę innych typów nanostruktur, takich jak kropki kwantowe lub zawiesiny koloidalne.

Kropki kwantowe (Rys. 3.5) to półprzewodnikowe kryształy o rozmiarach nanometrycznych, których właściwości optyczne i elektroniczne zależą bezpośrednio od ich wielkości, co wynika z efektu kwantowego. Koloidy (Rys. 3.6) z kolei to układy niejednorodne, w których cząstki jednej substancji (fazy rozproszonej) o wymiarach od 1 nm do 1000 nm są równomiernie rozprowadzone w drugiej substancji (fazie rozpraszającej), tworząc zawiesinę. Wykorzystywane są często do naniesienia nanostrutkur poprzez rozwirowanie koloidu na powierzchni oraz odparowanie fazy rozpraszającej.



Rys. 3.5: Kropki kwantowe na podłożu InGaAsN [2]



Rys. 3.6: Zdjęcie i odpowiadający mu obraz SEM kryształów koloidalnych. [6]

3.3. ImageJ

ImageJ to oprogramowanie wykorzystywane do analizy obrazów SEM w *Katedrze Mikroelektroniki i Nanotechnologii*. Program ten opracowany na zasadzie open-source przez *National Institutes of Health* jest jednym z najbardziej rozpowszechnionych narzędzi do badań naukowych wymagających przetwarzania obrazów wykonanych przy użyciu mikroskopów.

Warto nadmienić, że program *ImageJ* ma też unowocześnioną wersję *ImageJ2*, która wykorzystuje wszystkie rozwiązania oryginału, ale charakteryzuje się modułową strukturą pozwalającą na rozszerzenie funkcjonalności przez implementacje algorytmów wewnątrz środowiska.

PyImageJ

PyImageJ jest biblioteką, która umożliwia dostęp do funkcjonalności *ImageJ* bezpośrednio przy użyciu *python*.

Główną zaletą tego podejścia jest możliwość łączenia narzędzi przetwarzania obrazów *ImageJ* z ekosystemem bibliotek *python* przeznaczonych do analizy danych, obliczeń numerycznych oraz wizualizacji.
W niniejszej pracy zostały wykorzystane następujące biblioteki *Python*:

- **os** – biblioteka standardowa *python* służąca do operacji systemowych, związana z tworzeniem i zapisem plików;
- **ImageJ** – biblioteka *PyImageJ* umożliwiająca wykorzystanie środowiska *ImageJ* i dostęp do jego funkcji przetwarzania obrazów przy użyciu *python*;
- **scyjava (jimport)** – biblioteka zapewniająca integrację między *python* a JVM (Java Virtual Machine), umożliwiająca import i wykorzystanie klas *Java*; potrzebna do wywoływania niektórych funkcji wewnątrz *ImageJ*, których nie jest w stanie zrealizować biblioteka *PyImageJ*;
- **pandas** – biblioteka do analizy i manipulacji danymi liczbowymi, wykorzystywana do organizacji wyników pomiarów oraz eksportu danych do formatu *.csv*;
- **matplotlib.pyplot** – podstawowa biblioteka do tworzenia wykresów i wizualizacji danych w Pythonie;
- **seaborn** – biblioteka do rozszerzonej wizualizacji statystycznej oparta na bibliotece matplotlib;
- **PIL** – biblioteka do przetwarzania i wyświetlania obrazów, wykorzystywana do konwersji obrazów do formatu kompatybilnego z *tkinter*;
- **tkinter** – biblioteka do tworzenia GUI, graficzny interfejs użytkownika *(ang. Graphical User Interface)*;
- **OpenCV** – biblioteka do przetwarzania obrazów i wideo w czasie rzeczywistym.

4. Prace własne

4.1. Przygotowanie środowiska programistycznego

Środowisko programistyczne do pracy z biblioteką *PyImageJ* można skonfigurować na dwa podstawowe sposoby: poprzez menedżery pakietów np. *conda* lub za pomocą narzędzia *pip*, przy czym w niniejszej pracy wykorzystano *conda* jako narzędzie do zarządzania środowiskiem i zależnościami. Konfigurację przeprowadzono, podążając ściśle za oficjalną dokumentacją *PyImageJ* [7], aby zapewnić zgodność wersji i powtarzalność procesu instalacyjnego.

Wpierw należy dodać kanał *conda-forge*, który zapewnia dostęp do aktualnych pakietów open-source, oraz ustawić priorytet kanałów.

```
conda config --add channels conda-forge
conda config --set channel_priority strict
```

Utworzono dedykowane środowisko zawierające bibliotekę *PyImageJ* oraz *OpenJDK* w wersji 11.

```
conda create -n pyimagej pyimagej openjdk=11
```

Po zakończeniu instalacji aktywowane zostało przygotowane środowisko conda.

```
conda activate pyimagej
```

Weryfikację poprawności przeprowadzono poprzez uruchomienie krótkiego skryptu *python*, który importuje pakiet imagej, inicjalizuje *ImageJ* w zadanej wersji i wypisuje numer wersji.

```
python -c 'import ImageJ; ij = ImageJ.init("2.14.0");
print(ij.getVersion())'
```

Odpowiedzią powinno być : **2.14.0**

4.1.1. Inicjalizacja

```
ij = ImageJ.init()
```

Ten fragment kodu, inicjalizuje w aplikacji najnowszą dostępną wersję *ImageJ*. Odpowiednie parametry wewnątrz *()* pozwalają na doprecyzowanie preferowanej wersji. W tabeli 4.1 przedstawiono porównanie sposobów inicjalizowania *ImageJ*.

Tab. 4.1: Porównanie sposobów inicjalizacji *ImageJ* [7]

Wymaganie	Kod
Najnowsza dostępna wersja *ImageJ2*	`ij = ImageJ.init()`
Najnowsza dostępna wersja *ImageJ2* (bez oryginalnego ImageJ)	`ij = ImageJ.init(add_legacy=False)`
Konkretna wersja *ImageJ2*	`ij = ImageJ.init('2.14.0')`
Konkretna wersja *ImageJ2* (bez oryginalnego ImageJ)	`ij = ImageJ.init('2.14.0', add_legacy=False)`
Z GUI (najnowsza wersja)	`ij = ImageJ.init(mode='gui')`
Z GUI (konkretna wersja)	`ij = ImageJ.init('net.ImageJ:ImageJ:2.14.0', mode='gui')`
Z GUI (najnowsza wersja, tryb interaktywny)	`ij = ImageJ.init(mode='interactive')`
Z GUI (konkretna wersja, tryb interaktywny)	`ij = ImageJ.init('net.ImageJ:ImageJ:2.14.0', mode='interactive')`
Z wtyczkami *Fiji* (najnowsza wersja)	`ij = ImageJ.init('sc.fiji:fiji')`
Z wtyczkami *Fiji* (konkretna wersja)	`ij = ImageJ.init('sc.fiji:fiji:2.14.0')`
Z konkretną wtyczką (najnowsza wersja)	`ij = ImageJ.init(['net.ImageJ:ImageJ', 'net.preibisch:BigStitcher'])`
Z konkretną wtyczką (konkretna wersja)	`ij = ImageJ.init(['net.ImageJ:ImageJ:2.14.0', 'net.preibisch:BigStitcher:0.4.1'])`
Z lokalnego obrazu	`ij = ImageJ.init('/Applications/Fiji.app')`

Wszystkie inicjalizacje za wyjątkiem inicjalizacji z lokalnego obrazu wymagają połączenia z internetem.

4.2. Wczytanie obrazu

Pierwszym krokiem po inicjalizacji *ImageJ* jest wczytanie obrazu, który zostanie poddany analizie. Odbywa się to przy pomocy:

```
dataset = ij.io().open(input_image_path)
imp = ij.py.to_imageplus(dataset)
imp.show()
```

Użytkownik wskazuje obraz, który chce wczytać, a zmienna *input_image_path* przechowuje informacje o ścieżce prowadzącej do wskazanego obrazu. W kolejnym kroku obraz ten jest konwertowany do formatu *ImagePlus*, jest to format obsługiwany przez środowisko *ImageJ*, a następnie wyświetlany.

Konwersja do formatu *ImagePlus* pozwala na wykorzystywanie gotowych rozwiązań środowiska *ImageJ*, które będą potrzebne w dalszych fragmentach pracy.

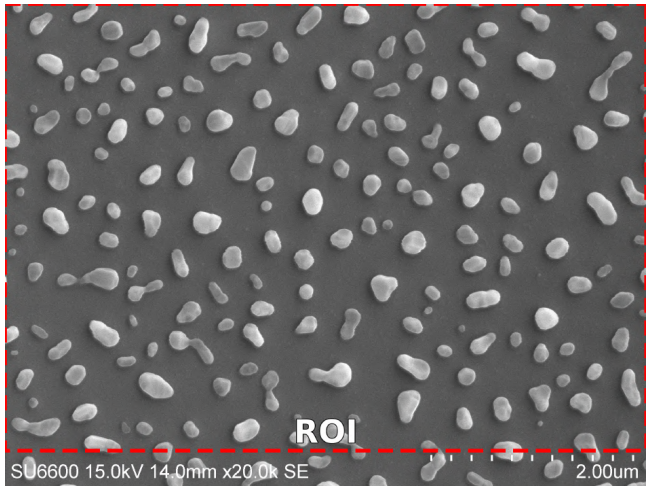
4.3. Kadrowanie

Obrazy wychodzące z mikroskopu *Hitachi SU6600* zawierają opis, znajdujący się w dolnej części, który należy usunąć przed przystąpieniem do analizy. Określając obszar roboczy jako 100% szerokości oraz 92% wysokości oryginalnego obrazu liczonej od góry, opis znajduje się poza wskazanym regionem niezależnie od rozdzielczości.

Proces kadrowania w aplikacji realizowany jest poprzez:

```
width = imp.getWidth()
height = imp.getHeight()
roi_width = width
roi_height = int(height * 0.92)

roi = Roi(0, 0, roi_width, roi_height)
imp.setRoi(roi)
imp.show()
```

Obszar roboczy (ROI, ang. *region of interest*) ustawiany jest w koordynatach (0,0) będącymi lewym górnym rogiem załadowanego obrazu (Rys. 4.1).



Rys. 4.1: Obszar roboczy na przykładowym obrazie z mikroskopu *Hitachi SU6600*

Realizacje przycięcia obrazu wykonujemy wywołując następującą funkcję:

```
ij.IJ.run(imp, "Crop", "")
```


4.4. Wstępna obróbka obrazu

Po otrzymaniu przyciętego obrazu należy przeprowadzić szereg operacji pozwalających na zmodyfikowanie go do postaci umożliwiającej przeprowadzenie analizy cząsteczek.

4.4.1. Wzmocnienie kontrastu

Wpierw obraz konwertowany jest do wersji z 8-bitową skalą szarości:

```
ij.IJ.run(imp, "8-bit", "")
```

Obrazy dostarczane przez mikroskop *Hitachi SU6600* są automatycznie w skali szarości, krok ten w tym przypadku jest opcjonalny. Jest on jednak zabezpieczeniem w przypadku wykorzystania obrazów, które nie są w skali szarości.

Wzmocnienie kontrastu w aplikacji odbywa się poprzez wywołanie:

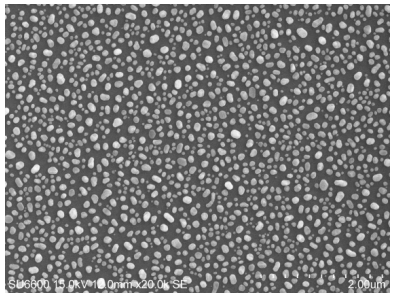
```
ij.IJ.run(imp, "Enhance Contrast", "saturated = 0.55")
```

Funkcja `Enhance Contrast` w *ImageJ* automatycznie normalizuje histogram obrazu poprzez rozciągnięcie zakresu intensywności pikseli [8, 9]. Algorytm przeskalowuje wartości pikseli P_{in} zgodnie z formułą:

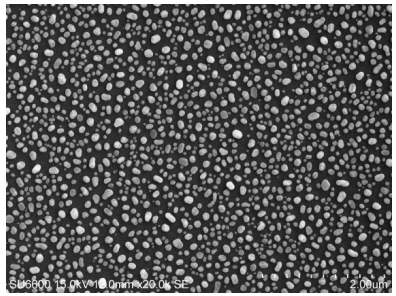
$$P_{\text{out}} = (P_{\text{in}} - c) \times \frac{b-a}{d-c} + a$$

gdzie:
$a = 0$, $b = 255$ – zakresy docelowe,
c , d – granice zakresu źródłowego wyznaczane na podstawie histogramu.

Parametr `saturated = 0.55` oznacza, że 0.55% najciemniejszych pikseli zostanie przypisane do wartość 0 (czarny), a 0.55% najjaśniejszych do wartości 255 (biały), natomiast pozostałe piksele rozciągnięte są równomiernie w całym dostępnym zakresie. Na rysunkach 4.2 i 4.3 przedstawiono efekt poprawy kontrastu.



Rys. 4.2: Obraz cząstek Pt na SiO2/GaN



Rys. 4.3: Obraz po poprawie kontrastu

4.4.2. Progowanie oraz binaryzacja

Aby poprawnie przeprowadzić analizę przy pomocy `Analyze Particles`, potrzeba obrazu składającego się tylko z czarnych i białych pikseli. Można to osiągnąć przy pomocy `setAutoThreshold` i `Convert to Mask`.

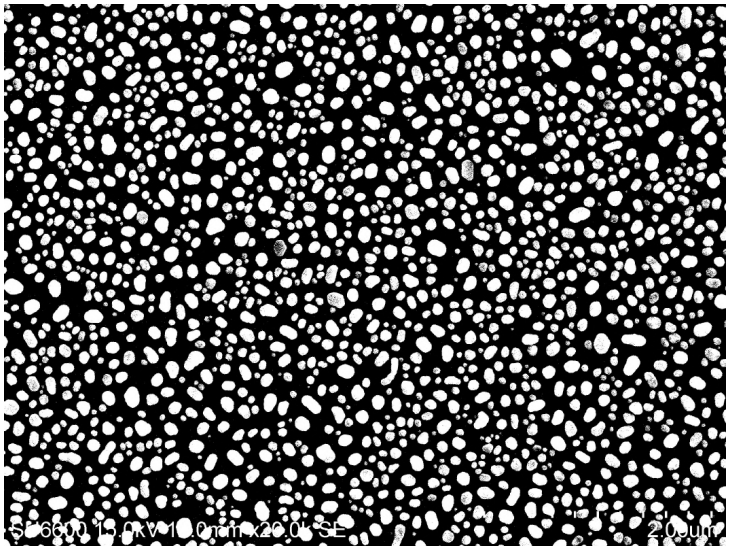
Funkcja `setAutoThreshold` automatycznie binaryzuje obraz za pomocą algorytmu *IsoData* [10].

Algorytm IsoData automatycznie wyznacza próg T poprzez iteracyjne szukanie wartości, która wyrównuje średnie intensywności pikseli ($I_{\text{śr ciemne}}, I_{\text{śr jasne}}$) poniżej i powyżej tego progu. Próg jest zdefiniowany jako:

$$T = \frac{I_{\text{śr ciemne}} + I_{\text{śr jasne}}}{2}$$

Po znalezieniu progu wszystkie piksele poniżej wartości T są mapowane na 0 (czarny), a piksele powyżej na 255 (biały), przy użyciu funkcji `Convert to Mask` (Rys. 4.4).

```
ij.IJ.setAutoThreshold(imp, "Default dark")
ij.IJ.run(imp, "Convert to Mask", "")
```

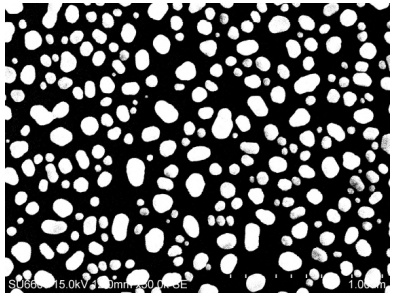


Rys. 4.4: Obraz po wywołaniu funkcji `Convert to Mask`

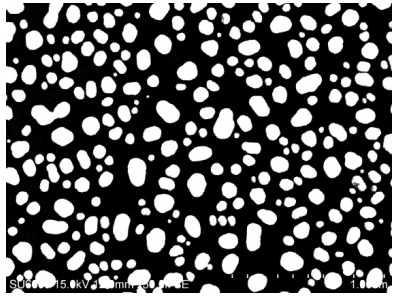
4.4.3. Wypełnienie dziur

Funkcja `Fill Holes` w *ImageJ* automatycznie wypełnia obszary („dziury”), które są zamknięte wewnątrz obiektów na obrazie. Algorytm analizuje każdy obiekt, identyfikując obszary czarnych pikseli w pełni otoczonych przez białe piksele, i zamienia wszystkie piksele należące do dziury na białe. Na rysunkach 4.5 i 4.6 przedstawiono zestawienie obrazów przed i po zastosowaniu funkcji `Fill Holes`.

```
ij.IJ.run(imp, "Fill Holes", "")
```



Rys. 4.5: Obraz przed `Fill Holes`



Rys. 4.6: Obraz po `Fill Holes`

4.5. Konfiguracja pomiaru

Gdy obraz jest już przygotowany do analizy, potrzebne jest określenie parametrów analizy.

```
ij.IJ.run("Set Measurements...", "area mean perimeter shape
feret's integrated centroid display redirect=None decimal=3")
```

4.5.1. Set Measurments

Przy pomocy polecenia `Set Measurments` można ustalić, które parametry będą obliczane, a później poddawane analize przy wywołaniu funkcji `AnalyzeParticles`:

- **Area** – pole powierzchni cząsteczki;
- **Mean** – średnia jasność (wartość szarości) wszystkich pikseli wewnątrz zaznaczonego obszaru;
- **Perimeter** – obwód obiektu;
- **Shape** – zestaw parametrów opisujących kształt obiektu; w jego skład wchodzą:
  - *Circularity* – miara zbliżenia kształtu do idealnego koła obliczana przy pomocy wzoru: $4\pi \cdot \frac{\textbf{Powierzchnia}}{\textbf{Obwód}^2}$; wartość 1 oznacza idealne koło;

— *Aspect Ratio* – stosunek długości osi głównej do osi podrzędnej elipsy opisującej obiekt: $\frac{\textbf{Oś główna}}{\textbf{Oś podrzędna}}$;
— *Roundness* – parametr odwrotny do współczynnika kształtu, uwzględniający pole powierzchni: $4 \cdot \frac{\textbf{Powierzchnia}}{\pi \cdot \textbf{Główna oś}^2}$;
— *Solidity* – stosunek pola powierzchni obiektu do pola jego otoczki wypukłej, czyli najmniejszy wielokąt wypukły, który zawiera dany zbiór punktów; określa gładkość i regularność brzegu obiektu; [11]

— **Feret's** – średnica Fereta definiowana jako największa odległość między dwoma punktami na obrysie;
— **Integrated** – iloczyn pola powierzchni i średniej jasności;
— **Centroid** – współrzędne geometrycznego środka obiektu.

Dodatkowo, z uwagi na zapotrzebowanie na taki parametr, pozwalający na łatwiejszą wizualizację rozmiaru niż w przypadku parametru powierzchni, zaimplementowane zostało również wyliczenie promienia (**Radius**), przy założeniu sferyczności struktury. Odbywa się to poprzez:

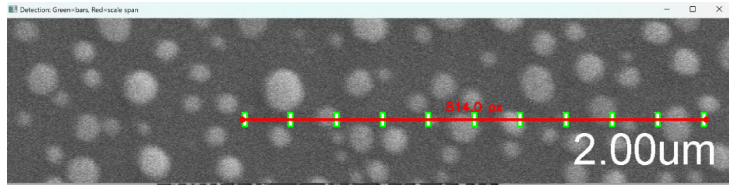
$$r = \sqrt{\frac{Powierzchnia}{\pi}}$$

4.5.2. setScale

Funkcja `setScale` odpowiada za ustawienie odpowiedniej skali tzn. przypisanie odpowiedniej ilości pikseli obrazu do fizycznego rozmiaru wyrażonego w nanometrach. W ramach prac nad aplikacją przetestowano dwa podejścia do realizacji tego zadania: automatyczne oraz manualne.

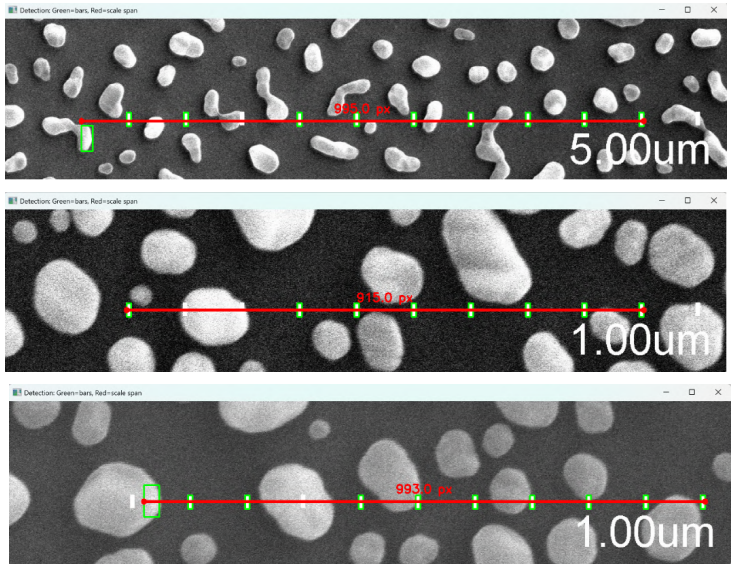
Podejście automatyczne (*OpenCV*)

W pierwszej kolejności podjęto próbę stworzenia algorytmu opartego na bibliotece *OpenCV*, który miał automatycznie lokalizować pasek skali. Algorytm opierał się na detekcji konturów i wyszukiwaniu w dolnej części obrazu pionowych, białych prostokątów stanowiących podziałkę oraz cyfry definiującej ilość mikrometrów (później przeliczanych na nanometry). Następnie wyznaczana była odległość między najbardziej skrajnie położonymi od siebie prostokątami w pikselach oraz wyliczany był współczynnik *px/nm* (Rys. 4.7).



Rys. 4.7: Poprawny odczyt podziałki

Mimo zastosowania filtracji geometrycznej (proporcje kształtu, minimalne i maksymalne pole powierzchni), metoda ta osiągnęła skuteczność jedynie na poziomie około 60%. Główną przyczyną błędów była trudność z rozróżnieniem elementu podziałki od białych (jasnych) elementów samego obrazu. Powyższe problemy przedstawiono na rysunku 4.8.



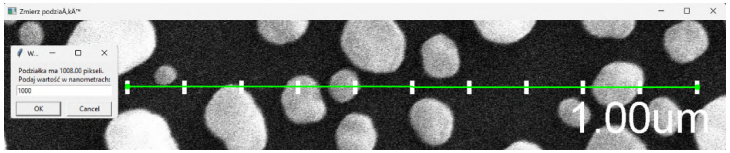
Rys. 4.8: Przykłady błędnych odczytów napotkanych podczas testów

Podejście manualne

Jako rozwiązanie docelowe wybrano metodę manualną. Przy użyciu `measure_scalebar_interactive`, wyświetlana jest wykadrowana część obrazu zawierającą podziałkę. Użytkownik ręcznie wskazuje dwa punkty: początek (x_1, y_1) oraz koniec (x_2, y_2) podziałki. Przy pomocy:

$$d_{px} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

wyznaczana jest odległość w pikselach, a następnie użytkownik wprowadza rzeczywistą wartość podziałki w nanometrach (d_{nm}). Na podstawie tych wartości obliczany jest współczynnik *px/nm* (Rys. 4.9).



Rys. 4.9: Poprawna kalibracja manualna

Metoda ta, choć wymaga ingerencji użytkownika, pozwala na jak najdokładniejsze wyznaczenie skali.

4.5.3. AnalyzeParticles

Narzędziem, umożliwiającym automatyczną detekcję i pomiar obiektów na obrazie binarnym w opracowanej aplikacji, jest funkcja `Analyze Particles` środowiska *ImageJ*. Algorytm przeszukuje obraz piksel po pikselu, identyfikując zmiany wartości jasności pomiędzy tłem a obiektem. Po wykryciu konturu struktury wyznaczane są jej parametry geometryczne zgodnie z konfiguracją ustaloną w `Set Measurements`. Zmierzone obiekty są oznaczane, co eliminuje ich wielokrotne zliczanie [8].

Wywołanie funkcji w kodzie *python* przyjmuje postać:

```
exclude_param = " exclude" if exclude_edges else ""
ij.IJ.run(imp, "Analyze Particles...",
f"size={size_parameter} circularity=0.00-1.00
show=Nothing display clear{exclude_param}")
```

`Exclude_param` został zaimplementowany w celu umożliwienia użytkownikowi nieuwzględniania w analizie elementów, których kontury mają kontakt z granicami obrazu, co mogłoby zaburzyć wyniki analizy rozmiaru i kształtu struktur.

4.6. Wyniki analizy

Po przeprowadzeniu `Analyze Particles` wyniki zapisywane są w pliku *.csv*, który zawiera komplet parametrów, opisujących wykryte cząsteczki, ustalone wewnątrz funkcji `Set Measurments` (Rys. 4.10).

```
results_table = ij.ResultsTable.getResultsTable()
csv_output = os.path.join(output, "wynik_csv.csv")
results_table.save(csv_output)
```

Rys. 4.10: Przykładowy plik .csv z parametrami ustalonymi przez `Set Measurements`

W kolejnym etapie plik *.csv* jest wczytywany do struktury `DataFrame` biblioteki *pandas*, co umożliwia łatwy dostęp do poszczególnych kolumn oraz ich bezpośrednie wykorzystanie przy tworzeniu wykresów.

```
df = pd.read_csv(csv_output)
```

Prócz pliku *nazwapliku_wynik_csv.csv*, generowany jest również plik z informacjami statystycznymi poszczególnych parametrów *nazwapliku_wynik_stat.csv* (Rys. 4.11).

Aplikacja automatycznie oblicza i zapisuje podstawowe statystyki analizowanych elementów. Wygenerowany plik z informacjami statystycznymi zawiera następujące parametry:

- **Statystyki powierzchni** – średnia, mediana oraz odchylenie standardowe pola powierzchni cząstek wyrażone w nm^2;
- **Statystyki promienia** – średni promień, mediana promienia oraz odchylenie standardowe wyrażone w nm;
- **Statystyki okrągłości** – średnia wartość parametru *Roundness*, mediana oraz odchylenie standardowe;

— **Pokrycie powierzchni** – procentowy udział powierzchni zajmowanej przez wykryte cząstki w stosunku do całkowitej przeanalizowanej powierzchni obrazu;
— **Liczba cząstek** – całkowita liczba wykrytych i zmierzonych struktur;
— **Gęstość powierzchniowa** – liczba cząstek na μm^2.

Obliczenia realizowane są przy użyciu funkcji statystycznych biblioteki *pandas*:

```
stats['Średnia powierzchnia (nm²)'] = df['Area'].mean()
stats['Mediana powierzchni (nm²)'] = df['Area'].median()
stats['Odchylenie standardowe powierzchni (nm²)'] = df['Area'].std()
```

Wyliczenie pokrycia wymaga wyznaczenia całkowitego pola przeanalizowanego obszaru. W tym celu aplikacja korzysta z wymiarów obrazu po kadrowaniu oraz wcześniej określony współczynnik konwersji pikseli na nanometry.

```
=== Statystyki ===
Średnia powierzchnia (nm²): 9074.6078
Mediana powierzchni (nm²): 8141.3510
Odchychlenie standardowe powierzchni (nm²): 3285.8182
Średni promień (nm): 52.9217
Mediana promienia (nm): 50.9065
Odchychlenie standardowe promienia (nm): 9.4108
Średnia okrągłość: 0.4304
Mediana okrągłości: 0.4290
Odchychlenie standardowe okrągłości: 0.0650
Pokrycie (%): 24.7757
Liczba cząstek: 121.0000
Gęstość powierzchniowa (cząstek/µm²): 27.3022
```

Rys. 4.11: Przykładowe wyniki analizy statystycznej

4.6.1. Graficzne przedstawienie wyników

Dane z `DataFrame` są wykorzystywane do automatycznego generowania wykresów przy użyciu bibliotek *matplotlib* i *seaborn*.

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.histplot(df, x='Radius', kde=True, ax=axes[0, 0])
sns.histplot(df, x='Circ.',  kde=True, ax=axes[0, 1])
sns.scatterplot(df, x='Radius', y='Perim.', ax=axes[1, 0])
sns.histplot(df, x='Feret',  kde=True, ax=axes[1, 1])
```



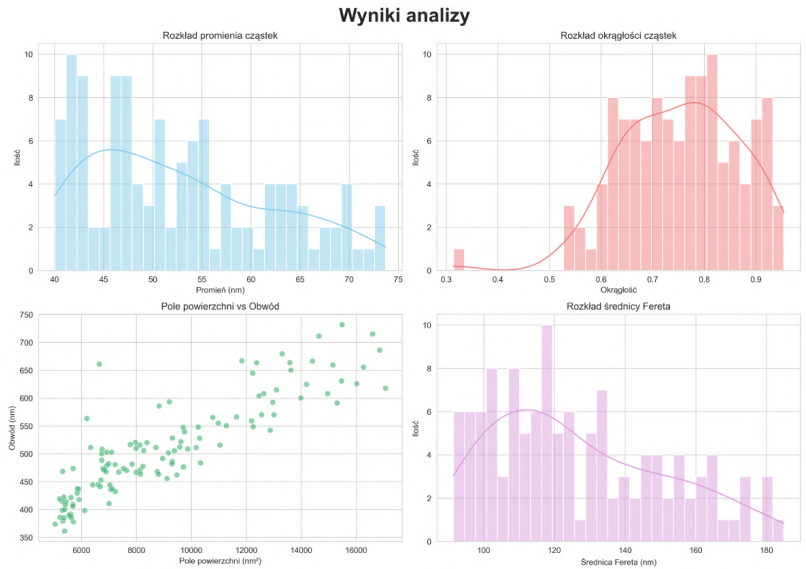
```
plt.savefig('{prefix}_wykresy.png',
dpi=300, bbox_inches='tight')


fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))

sns.scatterplot(df, x='Radius', y='Circ.', hue='Circ.', ax=axes2[0])

if 'AR' in df.columns:
    sns.histplot(df, x='AR', kde=True, ax=axes2[1])
elif 'Round' in df.columns:
    sns.histplot(df, x='Round', kde=True, ax=axes2[1])

plt.savefig('{prefix}_wykresy_dodatkowe.png',
dpi=300, bbox_inches='tight')
```

Dobór tych wykresów został dostosowany do zapotrzebowań *Katedry Mikroelektroniki i Nanotechnologii* i obejmuje:

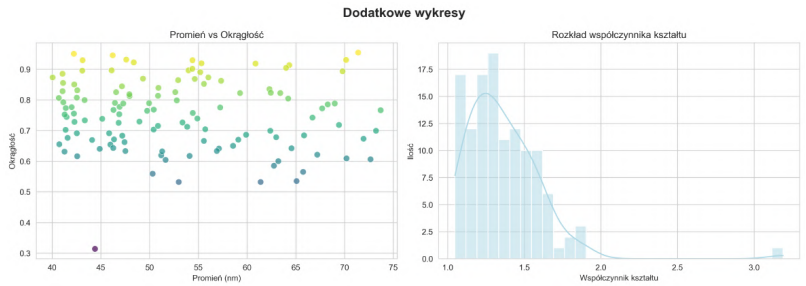
1. histogram rozkładu promienia cząstek,
2. histogram rozkładu okrągłości,
3. wykres powierzchnia–obwód,
4. histogram rozkładu średnicy Fereta,
5. wykres promień–okrągłość,
6. histogram współczynnika kształtu (`AR`).

Powstałe wykresy umożliwiają ocenę jednorodności rozmiaru cząstek, udziału struktur quasi-sferycznych i wydłużonych oraz zależności między rozmiarem a kształtem wysp. Histogramy rozkładów (promienia, okrągłości, średnicy Fereta, współczynnika kształtu) pozwalają identyfikować dominujące wielkości i morfologie oraz porównywać wyniki pomiędzy wieloma seriami eksperymentów. Wykresy rozrzutu (promień–obwód, promień–okrągłość) służą zarówno do weryfikacji spójności pomiarów geometrycznych, jak i do analizy korelacji między rozmiarem cząstek a ich kształtem.

Wyniki zapisywane są w dwóch plikach graficznych pod postacią: *nazwapliku_wykresy.png* (podstawowe rozkłady) oraz *nazwapliku_wykresy_dodatkowe.png*. Przykładowe wykresy przedstawione są na rysunkach 4.12 i 4.13.

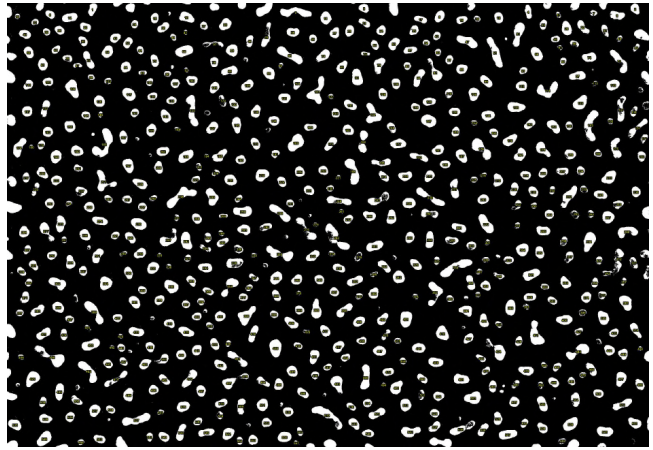


Rys. 4.12: Przykładowy plik *wykresy.png*



Rys. 4.13: Przykładowy plik *wykresy_dodatkowe.png*

Prócz wykresów, aplikacja generuje również obraz, na którym przeanalizowane elementy mają nałożone etykiety, bazując na ich centroidach. Umożliwia to użytkownikowi identyfikacje i analizę poszczególnych obiektów z osobna. (Rys. 4.14).

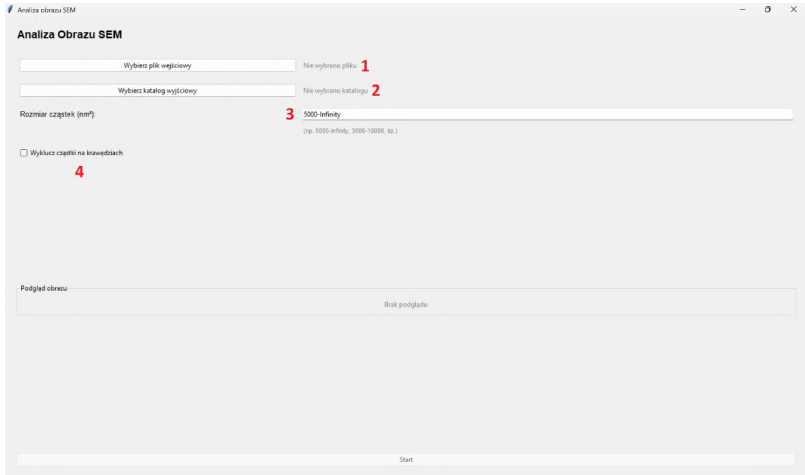


Rys. 4.14: Obraz z naniesionymi etykietami

4.7. Interfejs i obsługa aplikacji

Aplikacja została wyposażona w intuicyjny graficzny interfejs użytkownika zbudowany przy użyciu biblioteki *tkinter*. Okno główne zawiera podstawowe elementy umożliwiające przeprowadzenie analizy obrazu SEM.

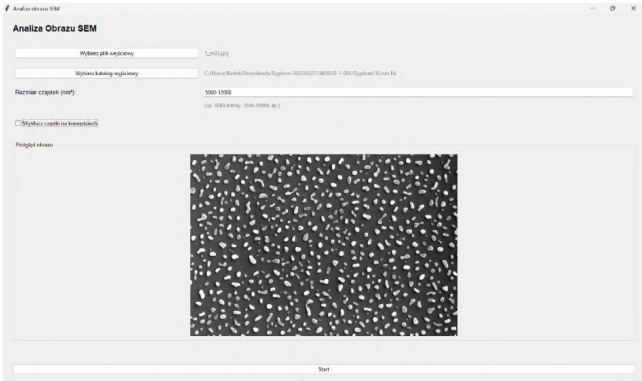
Po uruchomieniu programu użytkownik widzi okno aplikacji z czterema kluczowymi sekcjami (Rys. 4.15):



Rys. 4.15: Okno startowe aplikacji.

1. **Wybór pliku wejściowego** – przycisk *„Wybierz plik wejściowy"* otwiera okno dialogowe umożliwiające wskazanie obrazu SEM w formatach *.jpg*, *.png*, *.tif* lub *.tiff*. Po wyborze pliku jego nazwa wyświetlana jest obok przycisku.
2. **Wybór katalogu wyjściowego** – przycisk *„Wybierz katalog wyjściowy"* pozwala wskazać folder, w którym zostaną zapisane wszystkie pliki wynikowe (.csv, wykresy, obrazy).
3. **Parametr rozmiaru cząstek** – pole tekstowe umożliwia określenie zakresu pola powierzchni wykrywanych obiektów w nanometrach kwadratowych, które zostaną poddane analizie. Domyślnie ustawiony jest zakres `5000-Infinity`, z uwagi na wymaganie oprogramowania *ImageJ*.
4. **Opcja wykluczenia krawędzi** – pole wyboru *„Wyklucz cząstki na krawędziach"* pozwala odrzucić obiekty dotykające brzegu obrazu.

Po wybraniu pliku wejściowego w oknie pojawia się jego podgląd (Rys. 4.16), co umożliwia wstępną weryfikację jakości obrazu przed rozpoczęciem analizy.

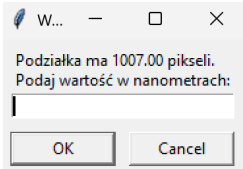


Rys. 4.16: Interfejs aplikacji z wczytanym obrazem wejściowym i wyświetlonym podglądem.

Po naciśnięciu przycisku *„Start"* aplikacja automatycznie wyświetla fragment obrazu zawierający podziałkę (prawy dolny róg), a użytkownik jest proszony o zmierzenie jej długości poprzez kliknięcie dwóch punktów na końcach paska skali (Rys. 4.17).

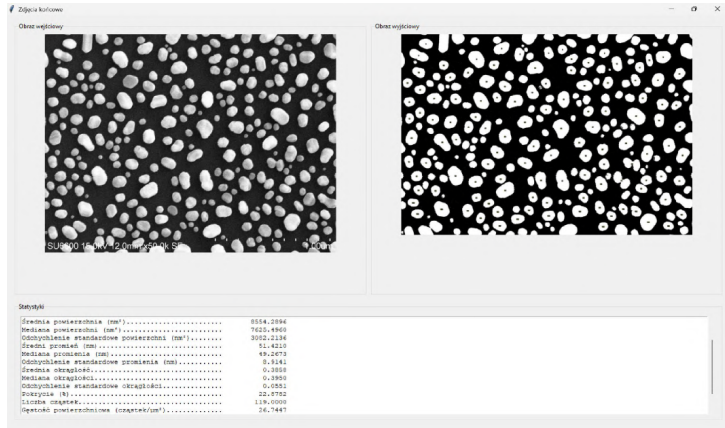
Rys. 4.17: Okno interaktywnego pomiaru podziałki.

Po zaznaczeniu punktów i zamknięciu okna użytkownik otrzymuje okno dialogowe z informacją o zmierzonej długości w pikselach oraz prośbą o podanie rzeczywistej wartości podziałki w nanometrach (Rys. 4.18).



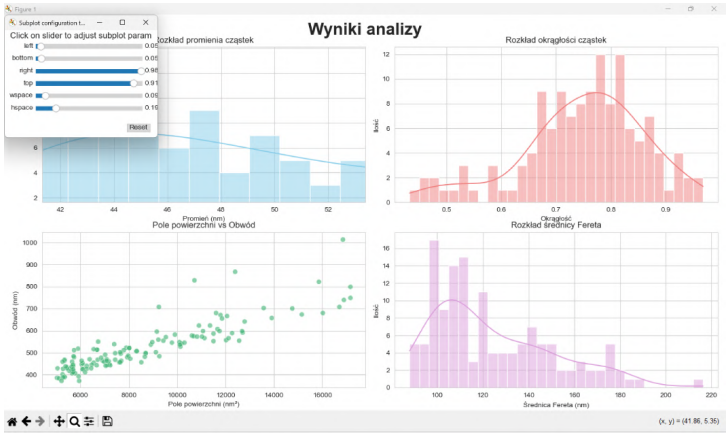
Rys. 4.18: Okno dialogowe do wprowadzenia rzeczywistej wartości podziałki w nanometrach.

Po zakończeniu analizy aplikacja automatycznie otwiera okno porównawcze (Rys. 4.19), przedstawiające zestawienie obrazów początkowego oraz końcowego. Poniżej nich znajdują się informacje statystyczne dotyczące analizowanego obrazu.



Rys. 4.19: Okno porównawcze wraz z informacjami statystycznymi

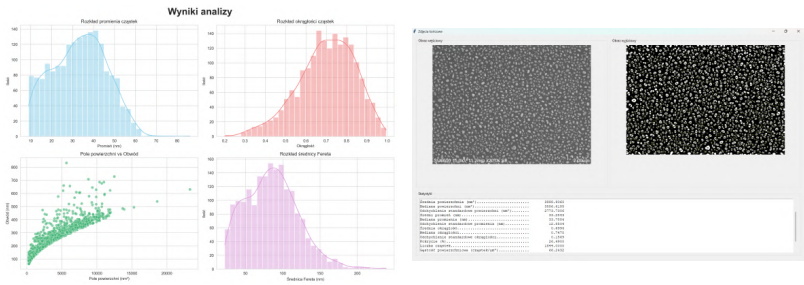
Dodatkowo aplikacja automatycznie wyświetla wygenerowane wykresy w osobnych oknach *matplotlib*, co pozwala na interaktywną eksplorację danych (powiększanie, przesuwanie, eksport do innych formatów) (Rys. 4.20).



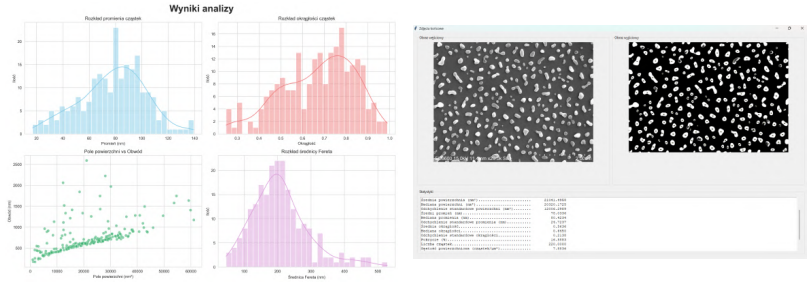
Rys. 4.20: Okno *matplotlib* do eksploracji wykresów

4.7.1. Praktyczne wykorzystanie aplikacji

W celu weryfikacji poprawności działania aplikacji przeprowadzono analizy na zestawie obrazów wykonanych podczas prac badawczych *Katedry Mikroelektroniki i Nanotechnologii*. Do analiz wykorzystano wyniki obrazowania próbek nanostrktur Pt na SiO_2/GaN (Rys. 4.21) oraz Ni na SiO_2/GaN (Rys. 4.22).



Rys. 4.21: Wyniki analizy obrazu SEM nanostrktur Pt na SiO_2/GaN



Rys. 4.22: Wyniki analizy obrazu SEM nanostrktur Ni na SiO_2/GaN

Uzyskane efekty analiz spełniają wymagania postawione przez cele pracy oraz użytkownika docelowego, dodatkowo rozszerzając niektóre funkcjonalności. Aplikacja została przetestowana na czterech zestawach komputerowych przez różnych użytkowników, otrzymując potwierdzenie jej użyteczności i intuicyjności obsługi.

5. Podsumowanie i wnioski

5.1. Podsumowanie

Celem niniejszej pracy było opracowanie aplikacji umożliwiającej automatyczną analizę statystyczną obrazów SEM samoorganizujących się nanostruktur metalicznych. Aplikacja została napisana w języku *python* z wykorzystaniem biblioteki *PyImageJ*, pozwalając na integrację bibliotek *python* do analizy statystycznej z analizą obrazu dostarczoną przez program *ImageJ*.

Główne osiągnięcia pracy obejmują:

— **Automatyzacja procesu analizy** – aplikacja realizuje pełny cykl przetwarzania obrazu, od wczytania pliku przez obróbkę wstępną (kadrowanie, wzmocnienie kontrastu, binaryzację) po detekcję i pomiar parametrów cząstek;
— **Charakteryzacja topologiczna** – aplikacja wyznacza 17 parametrów dla każdego wykrytego obiektu, w tym pole powierzchni, obwód, okrągłość, współczynnik kształtu, średnicę Fereta oraz współrzędne centroidu;
— **Automatyczna wizualizacja wyników** – generowane są wykresy statystyczne (histogramy rozkładów promienia, okrągłości, średnicy Fereta, współczynnika kształtu oraz wykresy rozrzutu promień–obwód i promień–okrągłość), które umożliwiają szybką interpretację wyników;
— **Interaktywna kalibracja skali** – zastosowanie manualnego pomiaru podziałki przyśpiesza proces i zapewnia wysoką dokładność;
— **Eksport danych do formatu .csv** – wszystkie wyznaczone parametry zapisywane są w formacie umożliwiającym dalszą obróbkę statystyczną w zewnętrznych narzędziach analitycznych.

Opracowane narzędzie spełnia zakładane cele pracy, stanowiąc funkcjonalną platformę do charakteryzacji nanostruktur sferycznych. Aplikacja została dostosowana do wymagań *Katedry Mikroelektroniki i Nanotechnologii*, zachowując jednocześnie potencjał do analizy szerokiej gamy struktur nanometrycznych oraz dalszej rozbudowy.

5.2. Możliwości rozwoju

Mimo osiągnięcia założonych celów oraz zrealizowania nadprogramowych funkcjonalności, aplikacja ma potencjał do dalszego rozwoju:

— **Udoskonalenie automatycznego wyznaczania skali** – w ramach pracy przetestowano algorytm oparty na detekcji konturów przy użyciu biblioteki *OpenCV*. Niesatysfakcjonująca precyzja działania wynikała z trudności w rozróżnieniu elementów podziałki od jasnych struktur na obrazie. Dalsze prace mogłyby się skupić na zastosowaniu technik uczenia maszynowego do rozpoznawania regionu podziałki oraz odczytu wartości liczbowych. Taka implementacja wyeliminowałaby konieczność manualnej interwencji użytkownika, zwiększając komfort pracy;
— **Przetwarzanie grupowe** – rozszerzenie aplikacji o możliwość analizy wielu obrazów na raz z automatycznym generowaniem zbiorczych raportów statystycznych oraz wykresów porównawczych między próbkami.

5.3. Znane błędy i problemy

W trakcie testowania aplikacji zidentyfikowano kilka ograniczeń oraz błędów:

— **Problem z wielokrotnym wczytywaniem obrazów** – w niektórych przypadkach aplikacja pozwala na załadowanie i przetworzenie obrazu tylko raz w danej sesji. Problem ten prawdopodobnie wynika z niepełnego zwalniania zasobów przez bibliotekę *PyImageJ* po zakończeniu przetwarzania. Obejściem problemu ponowne uruchomienie aplikacji przed analizą kolejnego obrazu;
— **Zależność od jakości obrazu wejściowego** – efektywność automatycznej analizy silnie zależy od kontrastu obrazu oraz jakości separacji cząstek od tła. Obrazy o niskim kontraście lub z nierównomiernym oświetleniem mogą prowadzić do błędnej detekcji;
— **Ograniczenia wykrywania struktur przylegających** – algorytm realizowany przez `Analyze Particles` może traktować dwie blisko położone cząstki jako pojedynczy obiekt, jeśli ich kontury stykają się po binaryzacji. W takich przypadkach parametry topologiczne (np. okrągłość, współczynnik kształtu) będą zniekształcone.

Kod źródłowy aplikacji został udostępniony na platformie *GitHub*, co pozwala zainteresowanym na dalszy rozwój i implementacje ulepszeń do aplikacji.

6. Dodatki

Spis rysunków

— Rys. 3.1 – Przykładowy obraz z mikroskopu SEM Hitachi SU6600 w trybie SE.
— Rys. 3.2 – Schemat zasady działania mikroskopu SEM.
— Rys. 3.3 – Formowanie się wysp metalicznych podczas SSD.
— Rys. 3.4 – Wyspa metaliczna w stanie równowagi z kątem θ.
— Rys. 3.5 – Przykładowy obraz kropek kwantowych (placeholder).
— Rys. 3.6 – Przykładowy obraz zawiesiny koloidalnej (placeholder).
— Rys. 4.1 – Obszar roboczy na przykładowym obrazie z mikroskopu Hitachi SU6600.
— Rys. 4.2 – Obraz oryginalny przed wzmocnieniem kontrastu.
— Rys. 4.3 – Obraz po poprawie kontrastu.
— Rys. 4.4 – Obraz po wywołaniu funkcji `Convert to Mask`.
— Rys. 4.5 - Obraz przed `Fill Holes`
— Rys. 4.6 – Obraz po zastosowaniu funkcji `Fill Holes`.
— Rys. 4.7 – Poprawny odczyt podziałki metodą *OpenCV*.
— Rys. 4.8 – Przykłady błędnych odczytów metodą *OpenCV*.
— Rys. 4.9 – Poprawna kalibracja manualna.
— Rys. 4.10 – Przykładowy plik *wynik_csv.csv*.
— Rys. 4.11 - Przykładowe wyniki analizy statystycznej
— Rys. 4.12 – Przykładowy plik *wykresy.png*.
— Rys. 4.13 – Przykładowy plik *wykresy_dodatkowe.png*.
— Rys. 4.15 – Okno startowe aplikacji.
— Rys. 4.16 – Interfejs aplikacji z wczytanym obrazem wejściowym.
— Rys. 4.17 – Okno interaktywnego pomiaru podziałki.
— Rys. 4.18 – Okno dialogowe do wprowadzenia wartości podziałki w nanometrach.
— Rys. 4.19 – Okno porównawcze: obraz wejściowy i obraz wynikowy.
— Rys. 4.20 - Okno *matplotlib* do eksploracji wykresów
— Rys. 4.21 - Wyniki analizy obrazu SEM nanostrktur Pt na SiO2/GaN
— Rys. 4.22 - Wyniki analizy obrazu SEM nanostrktur Ni na SiO2/GaN

Spis tabel

— Tab. 4.1 – Porównanie sposobów inicjalizacji *ImageJ*.

Pełny kod aplikacji

```
import os
import imagej
from scyjava import jimport
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import tkinter as tk
from tkinter import filedialog, messagebox, ttk, simpledialog
from PIL import Image, ImageTk, ImageDraw, ImageFont
import cv2
import numpy as np

input_image_path = None
output = None
process_status = False
size_parameter = "5000-Infinity"
scalebar_pixels = None
scalebar_nm = None
preview_photo = None
exclude_edges = False


def file_handler():
    global input_image_path, preview_photo
    path = filedialog.askopenfilename(
        title="Wybierz plik",
        filetypes=[("Image files", "*.jpg *.jpeg *.png *.tif *.tiff"), ("All files", "*.*")]
    )

    if path:
        input_image_path = path
        input_label.config(text=f"{os.path.basename(path)}")

        try:
            img = Image.open(path)
            img.thumbnail((600, 600))
            preview_photo = ImageTk.PhotoImage(img)
            preview_label.config(image=preview_photo, text="")
        except Exception as e:
            preview_label.config(text="Błąd wczytywania podglądu", image="")
            print(f"Preview error: {e}")

        check_ready()

def choose_output():
    global output
    path = filedialog.askdirectory(
        title="Wybierz, gdzie zapisać wyniki"
    )
    if path:
        output = path
        output_label.config(text=f"{path}")
        check_ready()

def check_ready():
    if input_image_path and output:
        start_button.config(state="normal")
    else:
        start_button.config(state="disabled")

def measure_scalebar_interactive(image_path):
    img_full = cv2.imread(image_path)
    height, width = img_full.shape[:2]

    crop_x = int(width * 0.50)
    crop_y = int(height * 0.88)

    img = img_full[crop_y:height, crop_x:width].copy()

    points = []

    def mouse_callback(event, x, y, flags, param):
        if event == cv2.EVENT_LBUTTONDOWN and len(points) < 2:
            points.append((x, y))
            cv2.circle(img, (x, y), 5, (0, 255, 0), -1)
            cv2.imshow('Zmierz podziałkę', img)

            if len(points) == 2:
```

```
                dist = np.sqrt((points[1][0] - points[0][0])**2 +
                               (points[1][1] - points[0][1])**2)
                print(f"Długość podziałki: {dist:.2f} pikseli")
                cv2.line(img, points[0], points[1], (0, 255, 0), 2)
                cv2.imshow('Zmierz podziałkę', img)

    cv2.imshow('Zmierz podziałkę', img)
    cv2.setMouseCallback('Zmierz podziałkę', mouse_callback)

    print("Kliknij początek i koniec podziałki w prawym dolnym rogu. Naciśnij ESC aby zakończyć.")
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    if len(points) == 2:
        distance = np.sqrt((points[1][0] - points[0][0])**2 +
                           (points[1][1] - points[0][1])**2)
        return distance
    return None

def add_particle_labels(image_path, csv_path, output_path):
    img = Image.open(image_path)
    draw = ImageDraw.Draw(img)


    df = pd.read_csv(csv_path)
    try:
        font = ImageFont.truetype("arial.ttf", 12)
    except:
        try:
            font = ImageFont.truetype("Arial.ttf", 12)
        except:
            font = ImageFont.load_default()

    for idx, row in df.iterrows():
        if 'X_pixels' in df.columns and 'Y_pixels' in df.columns:
            x = int(row['X_pixels'])
            y = int(row['Y_pixels'])
            label = str(idx + 1)

            bbox = draw.textbbox((0, 0), label, font=font)
            text_width = bbox[2] - bbox[0]
            text_height = bbox[3] - bbox[1]

            outline_width = 2

            for adj_x in range(-outline_width, outline_width + 1):
                for adj_y in range(-outline_width, outline_width + 1):
                    draw.text((x - text_width//2 + adj_x, y - text_height//2 + adj_y),
                              label, font=font, fill='black')

            draw.text((x - text_width//2, y - text_height//2),
                      label, font=font, fill='yellow')

    img.save(output_path)
    print(f"Labeled image saved to {output_path}")

def start_process():
    global process_status, size_parameter, scalebar_pixels, scalebar_nm, exclude_edges
    if not input_image_path or not output:
        messagebox.showerror("Błąd", "Proszę wybrać zarówno plik wejściowy, jak i katalog wyjściowy")
        return

    size_value = size_entry.get().strip()
    if not size_value:
        messagebox.showerror("Błąd", "Proszę podać parametr rozmiaru")
        return

    size_parameter = size_value
    exclude_edges = exclude_edges_var.get()

    start_button.config(state="disabled", text="...")
    root.update()

    try:
        scalebar_pixels = measure_scalebar_interactive(input_image_path)

        if scalebar_pixels is None:
            messagebox.showerror("Błąd", "Nie zmierzono podziałki")
            start_button.config(state="normal", text="Start")
            return
```



```
        scalebar_nm = float(simpledialog.askstring("Wartość podziałki",
                                                   f"Podziałka ma {scalebar_pixels:.2f} pikseli.\nPodaj wartość w nanometrach:"))

        if scalebar_nm is None or scalebar_nm <= 0:
            messagebox.showerror("Błąd", "Nieprawidłowa wartość podziałki")
            start_button.config(state="normal", text="Start")
            return

        process_images()
        start_button.config(state="normal", text="Start")
        final_images()
    except Exception as e:
        start_button.config(state="normal", text="Start")
        messagebox.showerror("Błąd", f"Wystąpił błąd podczas przetwarzania:\n{e}")

def final_images():
    prefix = os.path.splitext(os.path.basename(input_image_path))[0]

    comparison_window = tk.Toplevel(root)
    comparison_window.title("Zdjęcia końcowe")
    comparison_window.geometry("1400x700")
    frame = ttk.Frame(comparison_window, padding="10")
    frame.pack(fill=tk.BOTH, expand=True)

    try:
        input_img = Image.open(input_image_path)
        input_img.thumbnail((580, 550))
        input_photo = ImageTk.PhotoImage(input_img)

        input_frame = ttk.LabelFrame(frame, text="Obraz wejściowy", padding="5")
        input_frame.grid(row=0, column=0, padx=5, pady=5, sticky=(tk.W, tk.E, tk.N, tk.S))

        input_display = tk.Label(input_frame, image=input_photo)
        input_display.image = input_photo
        input_display.pack()
    except Exception as e:
        print(f"Error loading input image: {e}")

    try:
        jpeg_output_path = os.path.join(output, f"{prefix}_wynik_jpg.jpg")
        output_img = Image.open(jpeg_output_path)
        output_img.thumbnail((580, 550))
        output_photo = ImageTk.PhotoImage(output_img)

        output_frame = ttk.LabelFrame(frame, text="Obraz wyjściowy", padding="5")
        output_frame.grid(row=0, column=1, padx=5, pady=5, sticky=(tk.W, tk.E, tk.N, tk.S))

        output_display = tk.Label(output_frame, image=output_photo)
        output_display.image = output_photo
        output_display.pack()
    except Exception as e:
        print(f"Error loading output image: {e}")

    stats_frame = ttk.LabelFrame(frame, text="Statystyki", padding="10")
    stats_frame.grid(row=1, column=0, columnspan=2, padx=5, pady=10, sticky=(tk.W, tk.E, tk.N, tk.S))

    try:
        stats_csv_path = os.path.join(output, f"{prefix}_wyniki_stat.csv")
        stats_df = pd.read_csv(stats_csv_path)

        stats_text = tk.Text(stats_frame, height=12, width=120, font=('Courier', 10))
        stats_text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        scrollbar = ttk.Scrollbar(stats_frame, orient=tk.VERTICAL, command=stats_text.yview)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        stats_text.config(yscrollcommand=scrollbar.set)

        stats_text.insert(tk.END, "=" * 100 + "\n")
        stats_text.insert(tk.END, "STATYSTYKI ANALIZY\n")
        stats_text.insert(tk.END, "=" * 100 + "\n\n")

        for col in stats_df.columns:
            value = stats_df[col].iloc[0]
            stats_text.insert(tk.END, f"{col:.<50} {value:>15.4f}\n")

        stats_text.insert(tk.END, "\n" + "=" * 100 + "\n")
        stats_text.config(state=tk.DISABLED)

    except Exception as e:
```

```
        print(f"Error loading statistics: {e}")

    frame.columnconfigure(0, weight=1)
    frame.columnconfigure(1, weight=1)
    frame.rowconfigure(0, weight=1)
    frame.rowconfigure(1, weight=0)

def process_images():
    global scalebar_pixels, scalebar_nm
    print(f"Plik: {input_image_path}")
    print(f"Miejsce zapisu: {output}")

    prefix = os.path.splitext(os.path.basename(input_image_path))[0]

    pixels_per_nm = scalebar_pixels / scalebar_nm
    print(f"Skala: {pixels_per_nm:.4f} pikseli/nm")

    ij = imagej.init()

    Roi = jimport('ij.gui.Roi')

    dataset = ij.io().open(input_image_path)
    imp = ij.py.to_imageplus(dataset)
    imp.show()

    width = imp.getWidth()
    height = imp.getHeight()
    roi_width = width
    roi_height = int(height * 0.92)

    roi = Roi(0, 0, roi_width, roi_height)
    imp.setRoi(roi)
    imp.show()

    ij.py.run_macro("setBatchMode(true);")

    ij.IJ.run(imp, "Crop", "")
    ij.IJ.run(imp, "Enhance Contrast", "saturated=0.55")
    ij.IJ.run(imp, "8-bit", "")

    ij.IJ.setAutoThreshold(imp, "Default dark")
    ij.IJ.run(imp, "Convert to Mask", "")

    ij.IJ.run(imp, "Fill Holes", "")

    ij.IJ.run(imp,"Set Scale...", f"distance={scalebar_pixels} known={scalebar_nm} unit=nm")

    ij.IJ.run("Set Measurements...", "area mean perimeter shape feret's integrated centroid display redirect=None decimal=3")

    exclude_param = " exclude" if exclude_edges else ""
    ij.IJ.run(imp, "Analyze Particles...", f"size={size_parameter} circularity=0.00-1.00 show=Nothing display clear{exclude_param}")

    results_table = ij.ResultsTable.getResultsTable()
    csv_output = os.path.join(output, f"{prefix}_wynik_csv.csv")
    results_table.save(csv_output)

    jpeg_no_labels_path = os.path.join(output, f"{prefix}_wynik_bez_etykiet.jpg")
    ij.IJ.saveAs(imp, "Jpeg", jpeg_no_labels_path)

    df = pd.read_csv(csv_output)

    nm_per_pixel = scalebar_nm / scalebar_pixels
    if 'X' in df.columns and 'Y' in df.columns:
        df['X_pixels'] = df['X'] / nm_per_pixel
        df['Y_pixels'] = df['Y'] / nm_per_pixel

    csv_pixel_output_path = os.path.join(output, f"{prefix}_wynik_pixels.csv")
    df.to_csv(csv_pixel_output_path, index=False)

    jpeg_output_path = os.path.join(output, f"{prefix}_wynik_jpg.jpg")
    add_particle_labels(jpeg_no_labels_path, csv_pixel_output_path, jpeg_output_path)

    print("Koniec.")

    df = pd.read_csv(csv_output)

    if 'Area' in df.columns:
        df['Radius'] = np.sqrt(df['Area'] / np.pi)

    csv_output_with_radius = os.path.join(output, f"{prefix}_wynik_csv.csv")
```



```
    df.to_csv(csv_output_with_radius, index=False)

    print(f"\nAnalyzed {len(df)} particles")
    print(f"\nDataFrame columns: {df.columns.tolist()}")

    stats = {}

    if 'Area' in df.columns:
        stats['Średnia powierzchnia (nm²)'] = df['Area'].mean()
        stats['Mediana powierzchni (nm²)'] = df['Area'].median()
        stats['Odchychlenie standardowe powierzchni (nm²)'] = df['Area'].std()

    if 'Radius' in df.columns:
        stats['Średni promień (nm)'] = df['Radius'].mean()
        stats['Mediana promienia (nm)'] = df['Radius'].median()
        stats['Odchychlenie standardowe promienia (nm)'] = df['Radius'].std()

   if 'Round.' in df.columns:
        stats['Średnia okrągłość'] = df['Round.'].mean()
        stats['Mediana okrągłości'] = df['Round.'].median()
        stats['Odchychlenie standardowe okrągłości'] = df['Round.'].std()
    elif 'Circ.' in df.columns:
        stats['Średnia okrągłość'] = df['Circ.'].mean()
        stats['Mediana okrągłości'] = df['Circ.'].median()
        stats['Odchychlenie standardowe okrągłości'] = df['Circ.'].std()

    cropped_width = roi_width
    cropped_height = roi_height
    total_area_nm2 = (cropped_width / pixels_per_nm) * (cropped_height / pixels_per_nm)
    total_area_um2 = total_area_nm2 / 1e6

    if 'Area' in df.columns:
        total_particle_area = df['Area'].sum()
        coverage_percent = (total_particle_area / total_area_nm2) * 100
        stats['Pokrycie (%)'] = coverage_percent

    particle_count = len(df)
    stats['Liczba cząstek'] = particle_count
    stats['Gęstość powierzchniowa (cząstek/m²)'] = particle_count / total_area_um2

    stats_df = pd.DataFrame([stats])
    stats_csv_path = os.path.join(output, f"{prefix}_wyniki_stat.csv")
    stats_df.to_csv(stats_csv_path, index=False)
    print(f"\nStatystyki zapisane do: {stats_csv_path}")
    print("\n=== Statystyki ===")
    for key, value in stats.items():
        print(f"{key}: {value:.4f}")

    sns.set_style("whitegrid")

    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle('Wyniki analizy', fontsize=24, fontweight='bold')

    if 'Radius' in df.columns:
        sns.histplot(data=df, x='Radius', kde=True, bins=30, ax=axes[0, 0], color='skyblue')
        axes[0, 0].set_title('Rozkład promienia cząstek')
        axes[0, 0].set_xlabel('Promień (nm)')
        axes[0, 0].set_ylabel('Ilość')

    if 'Round' in df.columns:
        sns.histplot(data=df, x='Round', kde=True, bins=30, ax=axes[0, 1], color='lightcoral')
        axes[0, 1].set_title('Rozkład okrągłości cząstek')
        axes[0, 1].set_xlabel('Okrągłość')
        axes[0, 1].set_ylabel('Ilość')

    if 'Area' in df.columns and 'Perim.' in df.columns:
        sns.scatterplot(data=df, x='Area', y='Perim.', ax=axes[1, 0], color='mediumseagreen', s=50, alpha=0.6)
        axes[1, 0].set_title('Pole powierzchni vs Obwód')
        axes[1, 0].set_xlabel('Pole powierzchni (nm²)')
        axes[1, 0].set_ylabel('Obwód (nm)')

    if 'Feret' in df.columns:
        sns.histplot(data=df, x='Feret', kde=True, bins=30, ax=axes[1, 1], color='plum')
        axes[1, 1].set_title("Rozkład średnicy Fereta")
        axes[1, 1].set_xlabel("Średnica Fereta (nm)")
        axes[1, 1].set_ylabel('Ilość')

    plt.tight_layout()
    graphs_output_path = os.path.join(output, f'{prefix}_wykresy.png')
    plt.savefig(graphs_output_path, dpi=300, bbox_inches='tight')
```

```
    print(f"\nZapisano jako '{graphs_output_path}'")
    plt.show()

    fig2, axes2 = plt.subplots(1, 2, figsize=(14, 5))
    fig2.suptitle('Dodatkowe wykresy', fontsize=16, fontweight='bold')

    if 'Radius' in df.columns and 'Round' in df.columns:
        sns.scatterplot(data=df, x='Radius', y='Round', ax=axes2[0],
                        hue='Round', palette='viridis', s=60, alpha=0.7, legend=False)
        axes2[0].set_title('Promień vs Okrągłość')
        axes2[0].set_xlabel('Promień (nm)')
        axes2[0].set_ylabel('Okrągłość')

    if 'AR' in df.columns:
        sns.histplot(data=df, x='AR', kde=True, bins=30, ax=axes2[1], color='lightblue')
        axes2[1].set_title('Rozkład współczynnika kształtu')
        axes2[1].set_xlabel('Współczynnik kształtu')
        axes2[1].set_ylabel('Ilość')
    elif 'Round' in df.columns:
        sns.histplot(data=df, x='Round', kde=True, bins=30, ax=axes2[1], color='lightblue')
        axes2[1].set_title('Rozkład okrągłości')
        axes2[1].set_xlabel('Okrągłość')
        axes2[1].set_ylabel('Ilość')

    plt.tight_layout()
    additional_graphs_output_path = os.path.join(output, f'{prefix}_wykresy_dodatkowe.png')
    plt.savefig(additional_graphs_output_path, dpi=300, bbox_inches='tight')
    print(f"Zapisano jako '{additional_graphs_output_path}'")
    plt.show()

    print(df.describe())

root = tk.Tk()
root.title("Analiza obrazu SEM")
root.geometry("1200x800")
root.resizable(True, True)

main_frame = ttk.Frame(root, padding="20")
main_frame.grid(row=0, column=0, sticky=(tk.W, tk.E, tk.N, tk.S))

title_label = ttk.Label(main_frame, text="Analiza Obrazu SEM", font=("Helvetica", 14, "bold"))
title_label.grid(row=0, column=0, columnspan=2, pady=(0, 20), sticky=(tk.W, tk.E))


input_button = ttk.Button(main_frame, text="Wybierz plik wejściowy", command=file_handler)
input_button.grid(row=1, column=0, pady=10, padx=5, sticky=(tk.W, tk.E))

input_label = ttk.Label(main_frame, text="Nie wybrano pliku", foreground="gray")
input_label.grid(row=1, column=1, pady=10, padx=5, sticky=(tk.W, tk.E))

output_button = ttk.Button(main_frame, text="Wybierz katalog wyjściowy", command=choose_output)
output_button.grid(row=2, column=0, pady=10, padx=5, sticky=(tk.W, tk.E))

output_label = ttk.Label(main_frame, text="Nie wybrano katalogu", foreground="gray")
output_label.grid(row=2, column=1, pady=10, padx=5, sticky=(tk.W, tk.E))

size_label = ttk.Label(main_frame, text="Rozmiar cząstek (nm²):", font=("Helvetica", 10))
size_label.grid(row=3, column=0, pady=10, padx=5, sticky=tk.W)

size_entry = ttk.Entry(main_frame, width=30)
size_entry.insert(0, "5000-Infinity")
size_entry.grid(row=3, column=1, pady=10, padx=5, sticky=(tk.W, tk.E))

size_info = ttk.Label(main_frame, text="(np. 5000-Infinity, 3000-10000, itp.)", foreground="gray", font=("Helvetica", 8))
size_info.grid(row=4, column=1, pady=(0, 10), padx=5, sticky=(tk.W, tk.E))

exclude_edges_var = tk.BooleanVar(value=False)
exclude_edges_checkbox = ttk.Checkbutton(main_frame, text="Wyklucz cząstki na krawędziach", variable=exclude_edges_var)
exclude_edges_checkbox.grid(row=5, column=0, columnspan=2, pady=10, padx=5, sticky=tk.W)

preview_frame = ttk.LabelFrame(main_frame, text="Podgląd obrazu", padding="10")
preview_frame.grid(row=6, column=0, columnspan=2, pady=20, sticky=(tk.W, tk.E))

preview_label = ttk.Label(preview_frame, text="Brak podglądu", foreground="gray")
preview_label.pack()

start_button = ttk.Button(main_frame, text="Start", command=start_process, state="disabled")
start_button.grid(row=7, column=0, columnspan=2, pady=(30, 0), sticky=(tk.W, tk.E))

root.columnconfigure(0, weight=1)
```



```
root.rowconfigure(0, weight=1)

main_frame.columnconfigure(0, weight=1)
main_frame.columnconfigure(1, weight=2)
main_frame.rowconfigure(6, weight=1)

root.mainloop()
```

Kod dotyczący implementacji automatycznego wyznaczania skali przy pomocy OpenCV

```
def detect_scalebar_automatically(image_path):
    img_full = cv2.imread(image_path)
    if img_full is None:
        print("Błąd: Nie można wczytać obrazu")
        return None

    height, width = img_full.shape[:2]

    crop_x = int(width * 0.50)
    crop_y = int(height * 0.85)
    img_cropped = img_full[crop_y:height, crop_x:width].copy()

    gray = cv2.cvtColor(img_cropped, cv2.COLOR_BGR2GRAY)
    debug_img = img_cropped.copy()

    _, binary = cv2.threshold(gray, 220, 255, cv2.THRESH_BINARY)

    kernel = np.ones((2, 2), np.uint8)
    binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)

    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    print(f"\nZnaleziono {len(contours)} białych konturów w prawym dolnym rogu")

    vertical_bars = []
    for i, contour in enumerate(contours):
        x, y, w, h = cv2.boundingRect(contour)
        aspect_ratio = h / w if w > 0 else 0
        area = cv2.contourArea(contour)

        roi = gray[y:y+h, x:x+w]
        avg_brightness = np.mean(roi)

        if (aspect_ratio > 2 and
            avg_brightness > 200 and
            area > 30 and
            w >= 2 and w <= 100 and
            h > 10):

            vertical_bars.append({
                'x': x, 'y': y, 'w': w, 'h': h,
                'area': area,
                'aspect_ratio': aspect_ratio,
                'brightness': avg_brightness,
                'x_left': x,
                'x_right': x + w
            })

            cv2.rectangle(debug_img, (x, y), (x+w, y+h), (0, 255, 0), 2)

            print(f"   Pasek pionowy {len(vertical_bars)}: x={x} do {x+w}, y={y} do {y+h},
            rozmiar={w}x{h}px, proporcje={aspect_ratio:.2f}, jasność={avg_brightness:.1f}")
        else:
            print(f"   Odrzucono {i}: {w}x{h}px, proporcje={aspect_ratio:.2f}, jasność={avg_brightness:.1f}, obszar={area:.0f}")

    if not vertical_bars or len(vertical_bars) < 2:
        print(f"\nZnaleziono tylko {len(vertical_bars)} pasek/pasków pionowych. Potrzeba minimum 2 do pomiaru podziałki.")
        cv2.imshow('Detekcja: Zielone=paski pionowe', debug_img)
        print("Wyświetlanie wykrytych elementów przez 5 sekund...")
        cv2.waitKey(5000)
        cv2.destroyAllWindows()
        print("Przełączanie na pomiar manualny.")
        return None

    min_x = min(bar['x_left'] for bar in vertical_bars)
    max_x = max(bar['x_right'] for bar in vertical_bars)

    scalebar_length = max_x - min_x
    avg_y_center = int(np.mean([bar['y'] + bar['h']//2 for bar in vertical_bars]))
```

```
print(f"\n Wykryto {len(vertical_bars)} pasków pionowych")
print(f"  Lewy kraniec: x={min_x}")
print(f"  Prawy kraniec: x={max_x}")
print(f"  Rozpiętość pozioma (długość podziałki): {scalebar_length:.2f} pikseli")

cv2.line(debug_img, (min_x, avg_y_center), (max_x, avg_y_center), (0, 0, 255), 3)
cv2.circle(debug_img, (min_x, avg_y_center), 5, (0, 0, 255), -1)
cv2.circle(debug_img, (max_x, avg_y_center), 5, (0, 0, 255), -1)

cv2.line(debug_img, (min_x, avg_y_center - 5), (min_x, avg_y_center + 5), (0, 0, 255), 2)
cv2.line(debug_img, (max_x, avg_y_center - 5), (max_x, avg_y_center + 5), (0, 0, 255), 2)

cv2.putText(debug_img, f"{scalebar_length:.1f} px",
            ((min_x + max_x) // 2 - 50, avg_y_center - 15),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

cv2.imshow('Detekcja: Zielone=paski, Czerwone=rozpiętość', debug_img)
print("\nWyświetlanie wykrytych elementów przez 3 sekundy...")
cv2.waitKey(3000)
cv2.destroyAllWindows()

return scalebar_length
```

Bibliografia

[1] Joseph I. Goldstein, Dale E. Newbury, Joseph R. Michael, Nicholas W. M. Ritchie, John Henry J. Scott, and David C. Joy. *Scanning Electron Microscopy and X-Ray Microanalysis*. Springer, 4th edition, 2017.

[2] dr. inż. Andrzej Stafiniak.

[3] W. Zhou, R. Apkarian, Z. L. Wang, and D. Joy. *Fundamentals of Scanning Electron Microscopy (SEM)*. Springer, 2007.

[4] David Williams and C. Carter. *The Transmission Electron Microscope*, pages 3–22. 01 2009.

[5] Carl V. Thompson. Solid-state dewetting of thin films. *Annual Review of Materials Research*, 42, 2012.

[6] Cheng Fang, Songlin Yang, Xiaoqing Wang, Ping He, Ran Ye, and Yong-Hong Ye. Fabrication of two-dimensional silica colloidal crystals via a gravity-assisted confined self-assembly method. *Colloid and Interface Science Communications*, 37:100286, 07 2020.

[7] Curtis T. Rueden, Mark C. Hiner, Edward L. Evans, Michael A. Pinkert, Alice M. Lucas, Anne E. Carpenter, Beth A. Cimini, and Kevin W. Eliceiri. Pyimagej: A library for integrating imagej and python. *Nature Methods*, 19(11):1326–1327, Oct 2022.

[8] Rasband WS Ferreira T. Imagej user guide — ij 1.46, 2010—2012.

[9] Robert Fisher, Simon Perkins, Ashley Walker, and Erik Wolfart. Point operations – contrast stretching, 2003.

[10] T.W. Ridler and S. Calvard. Picture thresholding using an iterative selection method. *IEEE Transactions on Systems, Man, and Cybernetics*, 8(8), 1978.

[11] Dr. Hossein Arsham. Computational geometry and linear programs. *Journal of Mathematical Modelling and Algorithms in Operations Research*, 1:251–266, 04 1999.